



Control Statements: Part 1

C++ How to Program



4.1 Introduction

- ▶ Before writing a program to solve a problem, we must have a thorough understanding of the problem and a carefully planned approach to solving it.
- ▶ When writing a program, we must also understand the types of building blocks that are available and employ proven program construction techniques.
- ▶ In this chapter and in Chapter 5, Control Statements: Part 2; Logical Operators, we discuss these issues as we present the theory and principles of structured programming.



4.2 Algorithms

- ▶ Any solvable computing problem can be solved by the execution a series of actions in a specific order.
- ▶ An **algorithm** is **procedure** for solving a problem in terms of
 - the **actions** to execute and
 - the **order** in which the actions execute
- ▶ Specifying the or-der in which statements (actions) execute in a computer program is called **program control**.
- ▶ This chapter investigates program control using C++'s **control statements**.



4.3 Pseudocode

- ▶ **Pseudocode** (or “fake” code) is an artificial and informal language that helps you develop algorithms.
- ▶ Similar to everyday English
- ▶ Convenient and user friendly.
- ▶ Helps you “think out” a program before attempting to write it.
- ▶ Carefully prepared pseudocode can easily be converted to a corresponding C++ program.
- ▶ Normally describes only **executable statements**.
- ▶ Declarations (that do not have initializers or do not involve constructor calls) are not executable statements.
- ▶ Fig. 4.1 corresponds to the algorithm that inputs two integers from the user, adds these integers and displays their sum.



-
- 1 *Prompt the user to enter the first integer*
 - 2 *Input the first integer*
 - 3
 - 4 *Prompt the user to enter the second integer*
 - 5 *Input the second integer*
 - 6
 - 7 *Add first integer and second integer, store result*
 - 8 *Display result*
-

Fig. 4.1 | Pseudocode for the addition program of Fig. 2.5.



4.4 Control Structures

- ▶ Normally, statements in a program execute one after the other in the order in which they're written.
 - Called **sequential execution**.
- ▶ Various C++ statements enable you to specify that the next statement to execute may be other than the next one in sequence.
 - Called **transfer of control**.
- ▶ All programs could be written in terms of only three **control structures**
 - the **sequence structure**
 - the **selection structure** and
 - the **repetition structure**
- ▶ When we introduce C++'s implementations of control structures, we'll refer to them in the terminology of the C++ standard document as “control statements.”



4.4 Control Structures (cont.)

- ▶ Unless directed otherwise, the computer executes C++ statements one after the other in the order in which they're written—that is, in sequence.
- ▶ The Unified Modeling Language (UML) **activity diagram** of Fig. 4.2 illustrates a typical sequence structure in which two calculations are performed in order.
- ▶ C++ allows us to have as many actions as we want in a sequence structure.

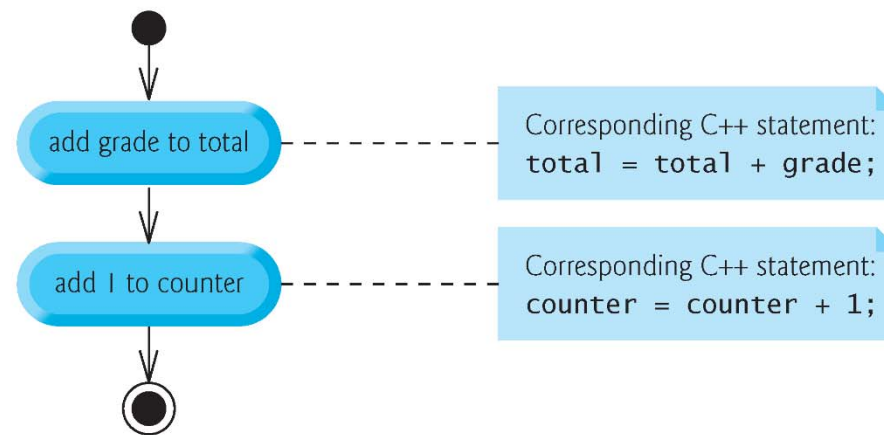


Fig. 4.2 | Sequence-structure activity diagram.



4.4 Control Structures (cont.)

- ▶ C++ provides three types of selection statements (discussed in this chapter and Chapter 5).
- ▶ The **if** selection statement either performs (selects) an action if a condition (predicate) is true or skips the action if the condition is false.
- ▶ The **if...else** selection statement performs an action if a condition is true or performs a different action if the condition is false.
- ▶ The **switch** selection statement (Chapter 5) performs one of many different actions, depending on the value of an integer expression.



4.4 Control Structures (cont.)

- ▶ The `if` selection statement is a **single-selection statement** because it selects or ignores a *single action* (or, as we'll soon see, a *single group of actions*).
- ▶ The `if...else` statement is called a **double-selection statement** because it selects between two different actions (or groups of actions).
- ▶ The `switch` selection statement is called a **multiple-selection statement** because it selects among many different actions (or groups of actions).



4.4 Control Structures (cont.)

- ▶ C++ provides three types of repetition statements (also called **looping statements** or **loops**) for performing statements repeatedly while a condition (called the **loop-continuation condition**) remains true.
- ▶ These are the **while**, **do...while** and **for** statements.
- ▶ The **while** and **for** statements perform the action (or group of actions) in their bodies zero or more times.
- ▶ The **do...while** statement performs the action (or group of actions) in its body *at least once*.



4.4 Control Structures (cont.)

- ▶ Each of the words `if`, `else`, `switch`, `while`, `do` and `for` is a C++ keyword.
- ▶ These words are reserved by the C++ programming language to implement various features, such as C++'s control statements.
- ▶ Keywords *cannot* be used as identifiers, such as variable names.
- ▶ Figure 4.3 provides a complete list of C++ keywords-.



C++ Keywords

Keywords common to the C and C++ programming languages

auto	break	case	char	const
continue	default	do	double	else
enum	extern	float	for	goto
if	int	long	register	return
short	signed	sizeof	static	struct
switch	typedef	union	unsigned	void
volatile	while			

C++-only keywords

and	and_eq	asm	bitand	bitor
bool	catch	class	compl	const_cast
delete	dynamic_cast	explicit	export	false
friend	inline	mutable	namespace	new
not	not_eq	operator	or	or_eq
private	protected	public	reinterpret_cast	static_cast
template	this	throw	true	try

Fig. 4.3 | C++ keywords. (Part I of 2.)



C++ Keywords				
typeid	typename	using	virtual	wchar_t
xor	xor_eq			
<i>C++11 keywords</i>				
alignas	alignof	char16_t	char32_t	constexpr
decltype	noexcept	nullptr	static_assert	thread_local



Fig. 4.3 | C++ keywords. (Part 2 of 2.)



4.5 if Selection Statement

- ▶ Programs use selection statements to choose among alternative courses of action.
- ▶ The following pseudocode determines whether “student’s grade is greater than or equal to 60” is **true** or **false**.

If student’s grade is greater than or equal to 60

Print “Passed”

- If **true**, “Passed” is printed and the next pseudocode statement in order is “performed” (remember that pseudocode is not a real programming language).
- If **false**, the print statement is ignored and the next pseudocode statement in order is performed.



4.5 if Selection Statement (cont.)

- ▶ The preceding pseudocode *If* statement can be written in C++ as
 - `if (grade >= 60)
 cout << "Passed";`
- ▶ Figure 4.4 illustrates the single-selection `if` statement.
- ▶ The diamond or **decision symbol** indicates that a *decision* is to be made.
 - The workflow will continue along a path determined by the symbol's associated **guard conditions**, which can be true or false.
 - Each transition arrow emerging from a decision symbol has a guard condition in *square brackets* above or next to the transition arrow.
 - If a guard condition is true, the workflow enters the action state to which that transition arrow points.

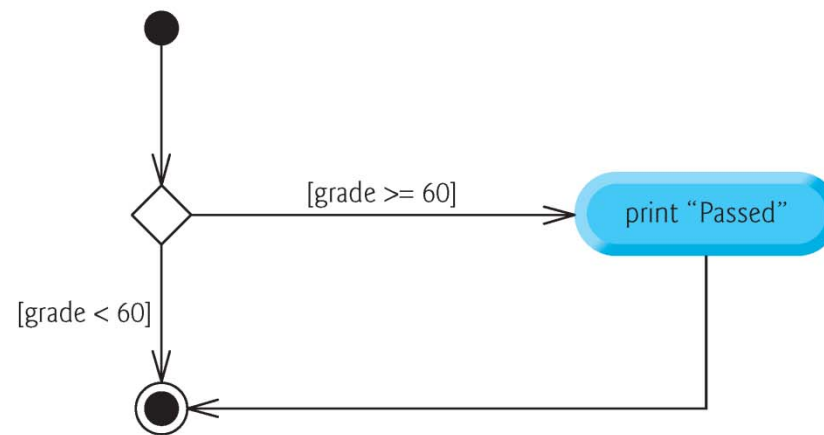


Fig. 4.4 | if single-selection statement activity diagram.



4.5 if Selection Statement (cont.)

- ▶ A decision can be based on any expression—if the expression evaluates to *zero*, it's treated as *false*; if the expression evaluates to *nonzero*, it's treated as true.
- ▶ C++ provides the data type **bool** for variables that can hold only the values **true** and **false**—each of these is a C++ keyword.



4.6 `if...else` Double-Selection Statement

- ▶ `if...else` double-selection statement
 - specifies an action to perform when the condition is true and a different action to perform when the condition is `false`.
- ▶ The following pseudocode prints “Passed” if the student’s grade is greater than or equal to 60, or “Failed” if the student’s grade is less than 60.

*If student’s grade is greater than or equal to 60
 Print “Passed”
Else
 Print “Failed”*



4.6 if...else Double-Selection Statement

- ▶ In either case, after printing occurs, the next pseudocode statement in sequence is “performed.”
- ▶ The preceding pseudocode *If...Else* statement can be written in C++ as

```
if ( grade >= 60 )  
    cout << "Passed";  
else  
    cout << "Failed";
```



4.6 `if...else` Double-Selection Statement (cont.)

- ▶ Figure 4.5 illustrates the the `if...else` statement's flow of control.

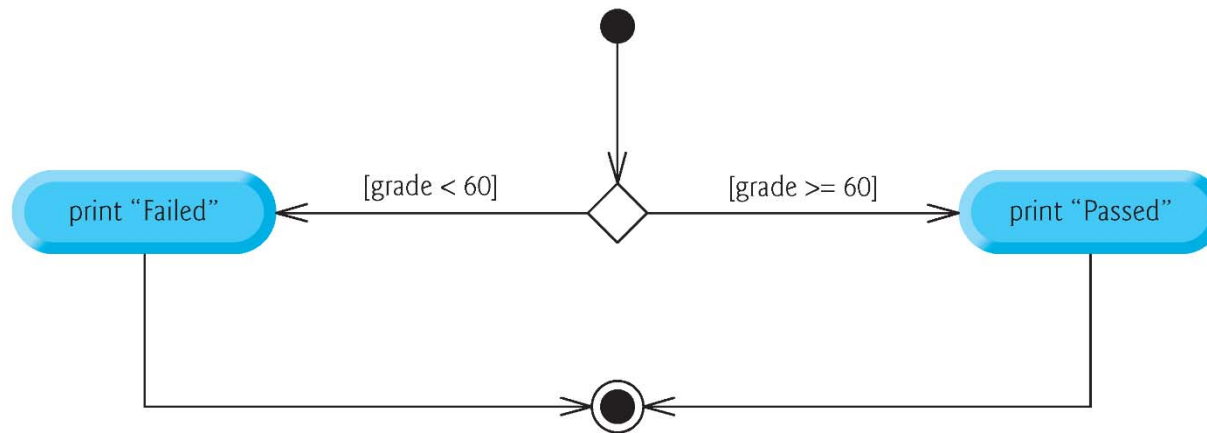


Fig. 4.5 | if...else double-selection statement activity diagram.



4.6 `if...else` Double-Selection Statement (cont.)

- ▶ **Conditional operator (`?:`)**
 - Closely related to the `if...else` statement.
- ▶ C++'s only **ternary operator**—it takes three operands.
- ▶ The operands, together with the conditional operator, form a **conditional expression**.
 - The first operand is a condition
 - The second operand is the value for the entire conditional expression if the condition is **true**
 - The third operand is the value for the entire conditional expression if the condition is **false**.
- ▶ The values in a conditional expression also can be actions to execute.



► Example:–

```
#include <iostream>
using namespace std;

int main ()
{
    int x, y = 10;

    x = (y < 10) ? 30 : 40;
    cout << "value of x: " << x << endl;

    return 0;
}
```



4.6 if...else Double-Selection Statement (cont.)

- ▶ **Nested if...else statements** test for multiple cases by placing **if...else** selection statements inside other **if...else** selection statements.

If student's grade is greater than or equal to 90

Print "A"

Else

If student's grade is greater than or equal to 80

Print "B"

Else

If student's grade is greater than or equal to 70

Print "C"

Else

If student's grade is greater than or equal to 60

Print "D"

Else

Print "F"



4.6 if...else Double-Selection Statement (cont.)

- ▶ This pseudocode can be written in C++ as

```
if ( studentGrade >= 90 ) // 90 and above gets "A"
    cout << "A";
else
    if ( studentGrade >= 80 ) // 80-89 gets "B"
        cout << "B";
    else
        if ( studentGrade >= 70 ) // 70-79 gets "C"
            cout << "C";
        else
            if ( studentGrade >= 60 ) // 60-69 gets "D"
                cout << "D";
            else // less than 60 gets "F"
                cout << "F";
```

- ▶ If studentGrade is greater than or equal to 90, the first four conditions are **true**, but only the output statement after the first test executes. Then, the program skips the **else**-part of the “outermost” **if...else** statement.



4.6 if...else Double-Selection Statement (cont.)

- ▶ Most programmers write the preceding statement as
 - ```
if (studentGrade >= 90) // 90 and above gets "A"
 cout << "A";
else if (studentGrade >= 80) // 80-89 gets "B"
 cout << "B";
else if (studentGrade >= 70) // 70-79 gets "C"
 cout << "C";
else if (studentGrade >= 60) // 60-69 gets "D"
 cout << "D";
else // less than 60 gets "F"
 cout << "F";
```
- ▶ The two forms are identical except for the spacing and indentation, which the compiler ignores.
- ▶ The latter form is popular because it avoids deep indentation of the code to the right, which can force lines to wrap.



### Performance Tip 4.1

A nested `if...else` statement can perform much faster than a series of single-selection `if` statements because of the possibility of early exit after one of the conditions is satisfied.



## 4.6 `if...else` Double-Selection Statement (cont.)

- ▶ The C++ compiler always associates an `else` with the immediately preceding `if` unless told to do otherwise by the placement of braces (`{` and `}`).
- ▶ This behavior can lead to what's referred to as the **dangling-else problem**.

```
• if (x > 5)
 if (y > 5)
 cout << "x and y are > 5";
 else
 cout << "x is <= 5";
```

appears to indicate that if `x` is greater than 5, the nested `if` statement determines whether `y` is also greater than 5.



## 4.6 if...else Double-Selection Statement (cont.)

- ▶ The compiler actually interprets the statement as
  - ```
if ( x > 5 )  
    if ( y > 5 )  
        cout << "x and y are > 5";  
    else  
        cout << "x is <= 5";
```
- ▶ To force the nested **if...else** statement to execute as intended, use:
 - ```
if (x > 5)
{
 if (y > 5)
 cout << "x and y are > 5";
}
else
 cout << "x is <= 5";
```
- ▶ Braces ({}) indicate that the second **if** statement is in the body of the first **if** and that the **else** is associated with the first **if**.



## 4.6 `if...else` Double-Selection Statement (cont.)

- ▶ The `if` selection statement expects only one statement in its body.
- ▶ Similarly, the `if` and `else` parts of an `if...else` statement each expect only one body statement.
- ▶ To include several statements in the body of an `if` or in either part of an `if...else`, enclose the statements in braces (`{` and `}`).
- ▶ A set of statements contained within a pair of braces is called a **compound statement** or a **block**.



► Example:–

```
if (x == 100)
{
 cout << "x is ";
 cout << x;
}
```



## 4.6 if...else Double-Selection Statement (cont.)

- ▶ Just as a block can be placed anywhere a single statement can be placed, it's also possible to have no statement at all—called a **null statement** (or an **empty statement**).
- ▶ The null state-ment is represented by placing a semicolon (;) where a statement would normally be.



## 4.6 if...else Double-Selection Statement (cont.)

```
include<iostream>
using namespace std;
void main()
{
 longint max,a,b,c;
 cout<< "\n Enter three integer";
 cin >> a,b,c;
 if ((a>b)&&(a>c))
 max=a;
 else if ((b>a)&&(b>c))
 max=b;
 else
 max =c;
 cout << "\n max no=" <<max;
}
```

► What is the output?